

Report on the Implementation of a Ray Tracer

Reagan Song

Bachelor of Computing Science, University of Technology Sydney

41052: Advanced Algorithms

Prof. Troy Lee

September 19, 2026

Overview

I chose ray tracing as my topic since Computer Graphics and Geometry are two of my main interests.

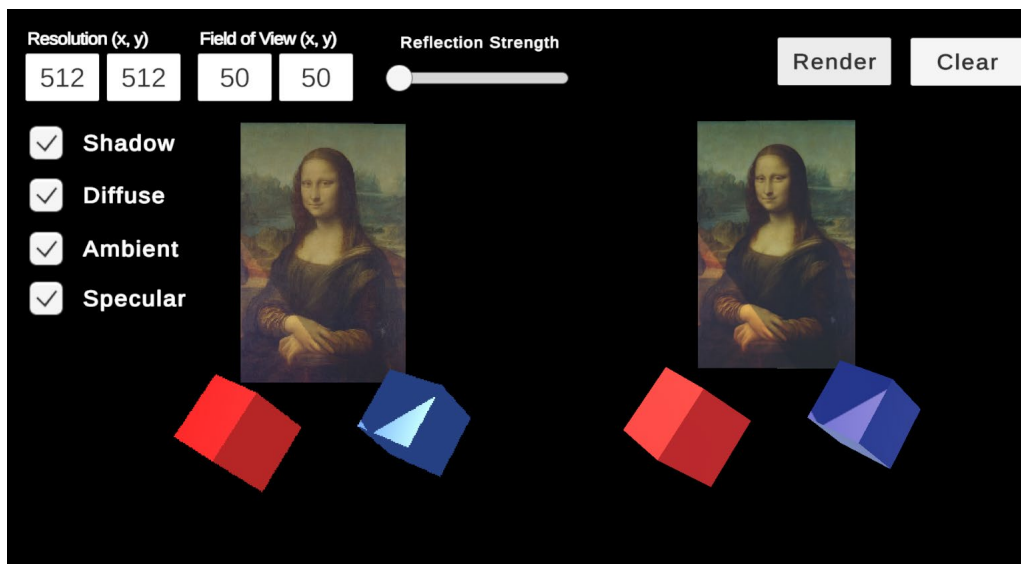
Using Unity Engine and C#, I implemented a simplified version of ray tracer that reproduces the pixels of game objects within a specified render region.

The program conducts custom ray tracing and shading/lighting through C# rather than Unity's standard rendering pipeline. It supports configurable render resolution, ambient, diffuse, and specular lighting, shadow calculation, and recursive reflection. After clicking the Render button, the program traces the game objects in the render region and displays the resulting image pixel by pixel.

Unity normally renders Lit Shader objects through its standard render pipeline. In this project, I used the Phong reflection model as the main reference to implement a simplified CPU-based ray tracer and shading system that produces an output as close as possible to the appearance of the same Lit Shader objects rendered by Unity.

Caption 1

Program interface: Custom ray-traced result (left) vs. Unity default rendering result (right)



Implementation

Tracing and Shading

I made a virtual camera which faces the game objects that I want to render. When the Render button is clicked, the program casts as many rays as the number of pixels in the selected resolution. For example, at 512×512 resolution, 262,144 rays are cast. These rays spread outward from the camera in a pyramid-like shape—also known as frustum. The direction of each ray was calculated by remapping the pixel coordinates from the image resolution to a normalized range of -1 to 1 around the image centre and then scaling them by the half-width and half-height of the frustum.

When a ray hits (detects) a game object, it accesses the texture image assigned to the object's material and obtains the UV coordinate corresponding to the hit point on the surface.

In Unity's Lit shader, the final base colour of a material is affected by both the texture colour and the material's tint colour—also known as albedo. Therefore, I used the same simplified idea and calculated the base colour by multiplying the texture colour by the albedo colour.

After calculating the base colour, the program checks whether the hit point is in shadow. It casts another ray from the hit point in the light direction. If this ray detects another object, it means the hit point is considered to be in shadow. This result is then used when calculating the diffuse and specular colour.

For the diffuse colour, I used the following simplified formula:

$$Diffuse = Albedo \times \text{Max}((N \cdot L), 0) \times LightColor$$

N means the normal vector of the hit point and L refers to the direction towards the light. For simplicity, I used only one directional light with the white colour of $(1,1,1)$. I also set the diffuse value to zero when in shadow. In the final implementation, I remapped the diffuse factor from 0–1 to 0–0.35 using Lerp to reduce the overall contrast.

For the ambient colour, Unity's Lit shader considers several factors such as ambient occlusion, global illumination (GI), and reflections. However, I used a much simpler approach:

$$Ambient = AmbientLightColor \times Albedo$$

In Unity, the ambient light colour can be configured separately, but in this project, I fixed it to white, (1, 1, 1) as well for simplicity. I then multiplied the ambient colour by an ambient strength of 0.6 in the final implementation.

Lastly, for the specular colour, I used a simplified form of the Phong specular model:

$$Specular = K_s \times \text{Max}((R \cdot V)^\alpha, 0) \times LightColor$$

K_s refers to the colour and strength of the reflected highlight. In Unity's Lit Shader, the specular response could be affected by several material properties, such as metallic values. However, I did not include those additional material properties in this project. In simpler shader models, the specular colour can also be provided directly as a material parameter, but since I did not use a custom shader, I simply fixed K_s to (1,1,1).

V is the direction from the hit point towards the camera, while R is the reflection direction of the incoming light around the normal. The dot product becomes larger when the camera direction is closer to the reflection direction, producing a stronger specular highlight. α is the shininess exponent, and I fixed to 16 since it is generally a user-defined parameter in material. A larger value produces a smaller and sharper highlight.

By adding the diffuse, ambient, and specular colours, we get the final colour. Then another ray is cast from the hit point in the surface's reflective direction of the current ray to detect any other objects. If one is hit, the ray tracing process is performed again for that object, and its calculated colour contributes to the reflected colour. This process is conducted recursively, with the maximum recursion depth limited to 3 for performance. Although the game objects in the scene do not become reflective enough to visibly mirror each other, I implemented reflection for experimental purpose.

Finally, the brightness and contrast values are applied to adjust the overall appearance of the result.

Drawing

Now the program has calculated one colour value for each pixel. The next step is to actually display

those colours. Instead of writing the result directly into a 2D texture, I decided to display each pixel using a small 3D cube.

Using nested for loops, the program creates a grid of cubes matching the selected resolution, with each cube corresponding to one pixel. The colour of each cube is then changed to the colour calculated during the ray tracing process. If no valid colour is obtained for a pixel, the program uses black as the default colour.

What I learned and Improved

Clamping Negative Values

Initially, I used the raw dot product formula regarding diffuse and specular calculations:

$Diffuse = Albedo \times (N \cdot L) \times LightColor$, $Specular = K_s \times (R \cdot V)^\alpha \times LightColor$, which can make negative numbers. Since negative lighting values would be subtracted from the final colour, I clamped them using $\max(0, \text{value})$.

Caption 2

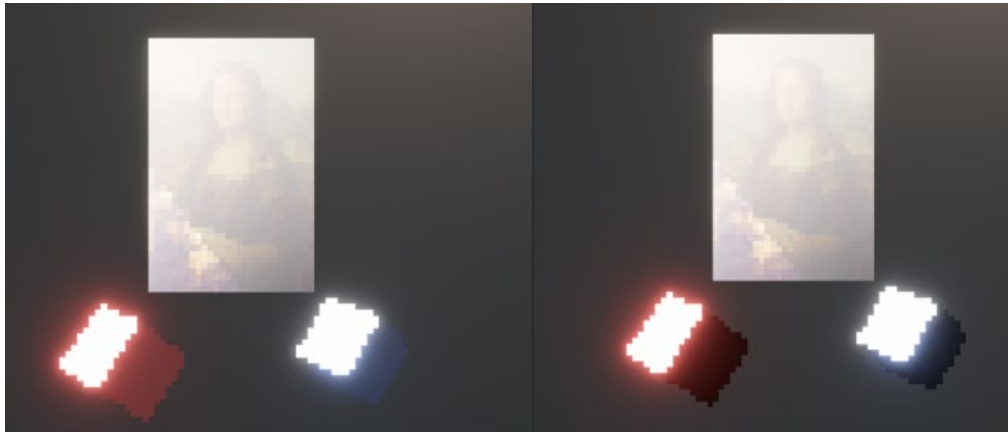
Diffuse comparison with 200×200 resolution, ignoring shadow (Clamped vs. Unclamped)



In the specular test, $R \cdot V$ sometimes reached approximately -0.7 to -0.9. This became more standing out when α was small, since the value was less reduced.

Caption 3

Specular comparison with 100×100 resolution, $\alpha = 1$, ignoring shadow (Clamped vs. Unclamped)



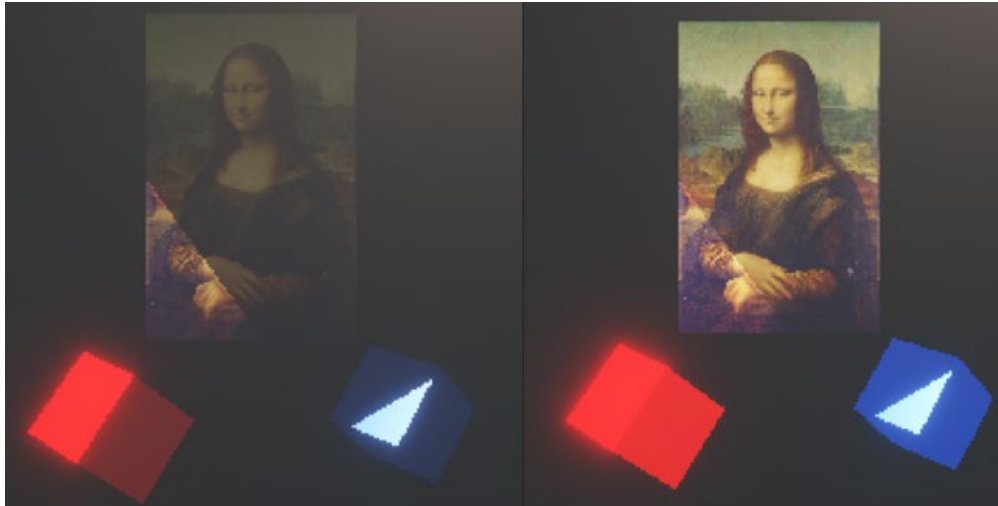
However, with shadow handling, removing the clamp produced little visible difference, since many of the affected areas were already assigned 0.

Handling Ambient Intensity

In the original ambient formula, the ambient colour was equal to the albedo colour because the ambient light colour was fixed to white for simplicity, and adding this value to the final colour made it too bright. Therefore, I multiplied the ambient colour by 0.3. This value was chosen simply to produce a result that looked closer to Unity's Lit Shader, rather than being derived from an additional lighting formula, since my goal was to keep the model as simple as possible.

Caption 4

Ambient lighting comparison (Scaling vs. Original)



Switching from Lit to Unlit

After finishing the implementation, I found out that I made a huge mistake: I had used a Lit shader instead of an Unlit shader for the generated cubes. This means that the cubes were affected by the directional light through Unity's Lit shader pipeline since the game objects' materials use Lit shader.

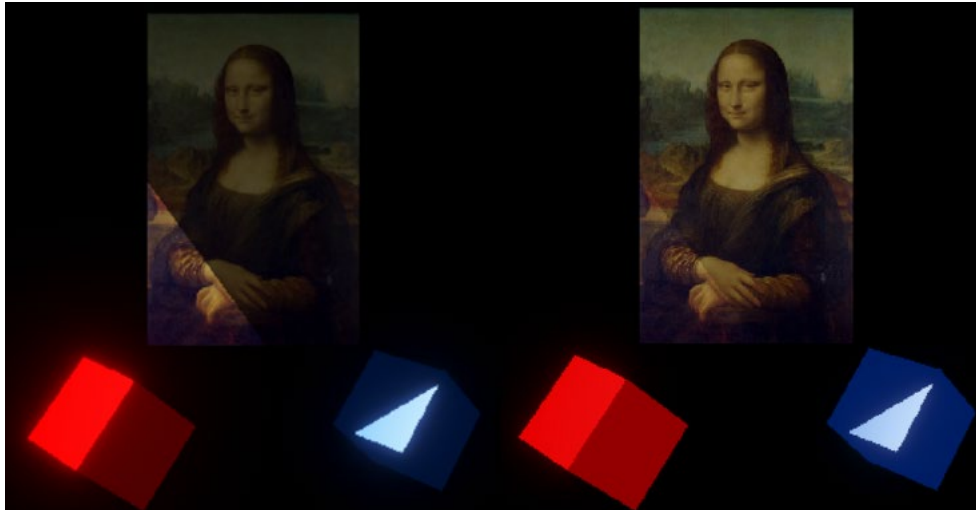
I immediately switched the material to an Unlit shader, which caused a significant change in both the overall brightness and contrast. To address this problem, I adjusted two factors; I changed the ambient intensity to 0.6 and applied a Lerp calculation to the diffuse factor.

Lerp is commonly used in shaders to remap values into a different range using $Lerp(a, b, t) = a + (b - a)t$. In this case, I used it to remap the diffuse factor range from 0-1 to 0-0.35—

$Lerp(0, 0.35, diffuse\ factor)$ —reducing the overall contrast of the result.

Caption 5

Unlit Shader comparison (Before vs. After Factors Adjustment)



3D Drawing Limitation

At first, I wrote the code that read each pixel colour of the texture and applied the colour to the Render Texture's pixels. However, I thought it would be more fun to visualize the drawing process in 3D, so I changed the code where it generated one small 3D cube for each pixel.

The main problem with this approach was performance. With the resolution of 500×500 the program already generates more than 250,000 game objects. This extremely increases draw calls and Unity also has to manage a very large number of objects every frame, which adds CPU overhead. As a result, the program became extremely laggy, and at the resolution of around 600-700, it froze.

However, 512×512 was already enough to produce a reasonably high-quality image and to compare each lighting factors, I addressed the problem by limiting the maximum resolution value to 512.

AI Use

I used Gemini to research existing lighting and shading formulas that I could use to imitate Unity's shader. I also asked Gemini to make a copyable table from my performance measurement results. However, I was not fully satisfied with using the old and simplified model. Even within the Phong model, there are more features I could have added but skipped.

I also asked ChatGPT to evaluate the change from modifying Render Texture pixels to visualizing

each pixel with a 3D cube. However, probably because I asked whether the idea was good for visualization rather than specifically asking about performance, it only praised the approach and did not mention any of its performance drawbacks.

Lastly, when implementing the reflection part, I asked Chat GPT about recursion depth, and it provided the basic structure of the reflection code. However, the suggested version called `TraceRay()` even when the reflection ray did not hit any object. In that case, `TraceRay()` returned the fallback black colour, which was then blended into the result and made the reflections appear darker overall.

I suppose that Gen AI can give very different answers depending on subtle differences in how a question is phrased, and it also tends to be less reliable for uncommon implementations with few existing examples in its training data, as it may be too focused on limited patterns rather than the broader context.